UNIVERSIDAD DE LOS LLANOS — INGENIERÍA DE SISTEMAS

Procesos Estocásticos (Código 603504) | Proyecto Final HMM 2D/3D

SUSTENTACIÓN MATEMÁTICA Y MAPEO AL CÓDIGO FUENTE

Guía de defensa oral en tablero: ecuaciones, demostraciones y su implementación exacta en NumPy

1. Guion de Sustentación Oral (Enunciado §5.2)

1. **Prueba de Trazo en Vivo (2D, 3D y Guía):** Mostrar `MainMenu`, trazar letras continuas (`a, v, n, m`) y multitrazo (`d, i, ñ, t, x`) en `View2D` observando la actualización de $P(c \mid O_{1:t})$ y la proyección verde; repetir en `View3D` dentro del recuadro azul de ±48°, rotar la esfera y mostrar `LetterGuideView` (ej. `d` en 2 trazos).
2. **Arquitectura Limpia y Cero Librerías Prohibidas:** Mostrar `requirements.txt` (solo `numpy` para cálculo numérico) y `core/HMMEngine.py` (0 librerías ML externas), además de la separación offline (`training/` → `models/*.json`) frente a la GUI.
3. **Defensa Matemática en Tablero:** Explicar los 4 pilares matemáticos y resolver el ejemplo numérico `N=2, T=2`.

2. Mapeo Ecuación ↔ Código Fuente

2.1 Discretización 2D y Geometría Diferencial en 3D (`core/FeatureExtractor.py`)

No usamos el tiempo del reloj (ms) porque la velocidad de dibujo deformaría la duración; parametrizamos por **longitud de arco constante** `D = 0.05`:

Ecuación Matemática	Implementación en Código
Normalización 2D: $y \leftarrow -y$, $s = \max(w, h, 250)$ en vivo o $\max(w, h, 10^{-6})$ al soltar, $p' = (p - p_{min}) / s$.	`FeatureExtractor2D.normalize()` y `View2D._evaluate_realtime()`
Suavizado simétrico bordes fijos (w=3): $\tilde{p}_i = (1/(2r_i+1)) \Sigma_{j=-r_i..r_i} p'_{i+j}$, $r_i = \min(1, i, n-1-i)$.	`FeatureExtractor._moving_average()` con `np.cumsum`
Remuestreo uniforme 2D: $s_i = \Sigma \lVert\tilde{p}_k - \tilde{p}_{k-1}\rVert_2$, $u_m = m \cdot D$, $q_m = \text{interp}(u_m, s, \tilde{p})$.	`FeatureExtractor2D.resample()`
Cuantización centrada (K=16 + 4 saltos): $o_m = \lfloor((\text{atan2}(\Delta y, \Delta x) + \pi/K) \bmod 2\pi) / (2\pi/K)\rfloor \bmod K$	`FeatureExtractor.quantize_direction()` y `quantize_jump()`
Descriptor morfológico 7D: $g = [\log(w/h), x'_{ini}, y'_{ini}, x'_{fin}, y'_{fin}, \bar{x}', \bar{y}']^T$.	`compute_shape_features()`
Ray-casting 3D y marco de cámara $R_{cam} \in SO(3)$: $\lVert o + t \cdot d\rVert^2 = R^2$; $\hat{p}_{cam} = R_{cam}(p/\lVert p\rVert)$ lleva la vista	`intersect_ray_sphere()`, `build_camera_matrix()`, `rotate_to_camera_frame()`

al ecuador (θ=π/2, ϕ=0).	
SLERP y rumbo tangente en S^2: $\hat{q}_m$ = [sin((1−t)Ω)$\hat{p}_k$ + sin(tΩ)$\hat{p}_{k+1}$] / sin Ω; ψ_m = atan2(−dθ_m, dϕ_m·sin $\bar{\theta}_m$).	`resample_geodesic()` y `directions_spherical()`

2.2 Topología Bakis y los 3 Problemas Fundamentales del HMM (`core/HMMEngine.py`)

Para cada letra c se define $\lambda_c = (A_c, B_c, \pi_c)$ con $N \in [4, 11]$ estados y topología **Bakis izquierda-derecha** ($\pi = [1, 0, \ldots, 0]^T$, $a_{ij} = 0$ si $j \notin \{i, i+1, i+2\}$), inicializada por segmentación uniforme en `init_segmental_bakis_hmm()`.

Problema HMM	Fórmula Clave para Tablero	Método en `HMMEngine.py`
1. Evaluación (Forward Escalado)	$\tilde{\alpha}_{t+1}(j) = [\Sigma_i \hat{\alpha}_t(i)a_{ij}]b_j(O_{t+1})$, $c_{t+1} = 1/\Sigma_j \tilde{\alpha}_{t+1}(j)$ Por inducción: $\hat{\alpha}_T(i) = (\Pi c_t)\alpha_T(i) \Rightarrow \log P(O\|\lambda) = -\Sigma_{t=1..T} \log(c_t)$.	`HMM.forward()`
2. Decodificación (Viterbi Log-Space)	$\delta_t(j) = \max_i[\delta_{t-1}(i) + \log a_{ij}] + \log b_j(O_t)$ Backtracking: $q_t^* = \psi_{t+1}(q_{t+1}^*)$ halla el estado actual q_T^*.	`HMM.viterbi()`
3. Aprendizaje (Baum-Welch EM)	Los factores c_t se cancelan exactamente en el Paso E: $\gamma_t(i) = \hat{\alpha}_t(i)\hat{\beta}_t(i)/c_t$, $\xi_t(i,j) = \hat{\alpha}_t(i)a_{ij}b_j(O_{t+1})\hat{\beta}_{t+1}(j)$. Paso M reestima A, B y aplica suavizado circular 0.78 / 0.11 / 0.11 en B[:, :K].	`HMM.backward()` y `HMM.fit()`

2.3 Clasificación Bayesiana Conjunta y Proyección Dinámica Futura

$$\text{Score}(c) = \log P(O_{1:t} \mid \lambda_c) - P_{dur}(t; \mu_c, \sigma_c) - P_{shape}(g) - P_{jump}(J_{obs}; J_c)$$

- **Duración (`length_deviation_penalty`):** $P_{dur} = 0.5\cdot((T - \mu_c)/\max(\sigma_c, 4))^2$ (en parciales solo si $T > \mu_c + 1.5\sigma_c$).
- **Morfología (`shape_penalty`):** $P_{shape} = w\cdot0.5\cdot\Sigma_{d=1..7}\ ((g_d - \mu_{shape,d})/\sigma_{shape,d})^2$ (desempata a vs o).
- **Saltos (`jump_penalty`):** penaliza discrepancia entre trazos continuos y multitrazo (separa v/l de x/i).
- **Log-Sum-Exp (`HMMClassifier.classify`):** P(c | O, g) = exp(Score(c) − m − log $\Sigma_{c'}$ exp(Score(c') − m)).
- **Proyección Futura (`project_future_symbols` + `step_geodesic`):** La permanencia en S_j es geométrica con media $E[d_j] = 1/(1 - a_{jj})$. En 2D avanza $\hat{p}_k = \hat{p}_{k-1} + D(\cos\phi, \sin\phi)$; en 3D usa el mapa exponencial $\hat{p}_k = \cos(\delta)\hat{p}_{k-1} + \sin(\delta)\hat{t}(\psi)$.

3. Ejemplo Numérico Corto para Tablero (N=2, T=2)

Dado $\pi = [1, 0]^T$, $A = [[0.6, 0.4], [0.0, 1.0]]$, $B = [[0.8, 0.2], [0.1, 0.9]]$ y secuencia $O = (0, 1)$:

- **Forward t=1 (O_1=0):** $\tilde{\alpha_1}(0) = 1 \times 0.8 = 0.8$, $\tilde{\alpha_1}(1) = 0 \Rightarrow c_1 = 1/0.8 = 1.25$, $\hat{\alpha_1} = [1.0, 0.0]^T$.
- **Forward t=2 (O_2=1):** $\tilde{\alpha_2}(0) = (1.0 \times 0.6) \times 0.2 = 0.12$, $\tilde{\alpha_2}(1) = (1.0 \times 0.4) \times 0.9 = 0.36 \Rightarrow$ suma 0.48, $c_2 = 1/0.48$, $\hat{\alpha_2} = [0.25, 0.75]^T$.
- **Verosimilitud Total:** $P(O \mid \lambda) = 1/(c_1 c_2) = 0.8 \times 0.48 = 0.384$, $\log P(O \mid \lambda) = -\log c_1 - \log c_2 = \log(0.384)$.
- **Viterbi y Proyección:** Camino $(S_0, S_0) = 0.096$ vs $(S_0, S_1) = 0.288 \Rightarrow Q^* = (S_0, S_1)$. En S_1 el símbolo dominante es $\text{argmax}_k\, b_1(k) = 1$.

4. Respuestas Rápidas a Preguntas Frecuentes del Docente

Pregunta del Docente	Respuesta Respaldada en el Código
¿Por qué no depende de la velocidad?	`resample()` y `resample_geodesic()` parametrizan por longitud de arco constante; los timestamps `[t]` no se usan en inferencia.
¿Cómo evitan singularidad en polos 3D?	`rotate_to_camera_frame()` rota la vista al ecuador frontal ($\theta=\pi/2$), `np.unwrap` elimina saltos en $\pm\pi$ y el rumbo usa $d\phi \cdot \sin(\theta)$.
¿Por qué un garabato final da 100%?	Bayes normaliza sobre 27 clases cerradas ($\Sigma\ P(c\|O) = 1$); si una clase supera a las demás por ≥20 log-unidades alcanza 100% relativo, aunque su `Score` bruto caiga a ≈ -173 (frente a ≈ -30 de una letra real).